

Fake Job Detection NLP

1. Introduction

The objective of this assignment is to implement, evaluate, and compare multiple deep learning architectures for Natural Language Processing (NLP). The selected task is Fake Job Posting Detection, framed as a binary classification problem (1 = Fraudulent/Fake, 0 = Real). The dataset utilized is the EMSCAD Fake Job Postings dataset, which contains approximately 17,880 records.

A primary challenge of this dataset is severe class imbalance; roughly 95% of the postings are real, while only 5% are fraudulent. The goal is to determine how classical sequential models (RNNs), convolutional models (CNNs), and modern self-attention frameworks (Transformers) process linguistic context to identify scams.

2. Methodology & Text Preprocessing

Before model training, the dataset underwent strict preprocessing to ensure high-quality numerical representation:

- **Feature Engineering:** Text-heavy columns (`title`, `company_profile`, `description`, `requirements`, and `benefits`) were concatenated into a single master text feature to provide maximum context.
- **Text Cleaning:** HTML tags, special characters, and excessive whitespaces were removed using regular expressions. All text was converted to lowercase.
- **Handling Imbalance:** Class weights were calculated and applied to the Cross-Entropy Loss functions during training to penalize the models more heavily for missing the minority "fake" class.
- **Tokenization:** Traditional models utilized standard word-level embeddings (vocab size = 30,000), the Character-level model utilized a customized character dictionary, and Transformer models utilized their respective HuggingFace subword tokenizers.
- **Hardware:** All models were trained and evaluated on a Google Colab T4 GPU (16 GB VRAM) using PyTorch.

3. Implemented Architectures

To perform a comprehensive comparative analysis, 10 distinct deep learning models were trained on the exact same 70/15/15 (Train/Val/Test) dataset split:

- Simple RNN
- LSTM
- GRU
- 1D CNN (TextCNN)
- BiLSTM
- LSTM with Attention
- Character-Level Deep Model (Char-CNN + LSTM)
- BERT (bert-base-uncased)
- T5 (t5-small)
- DistilGPT-2

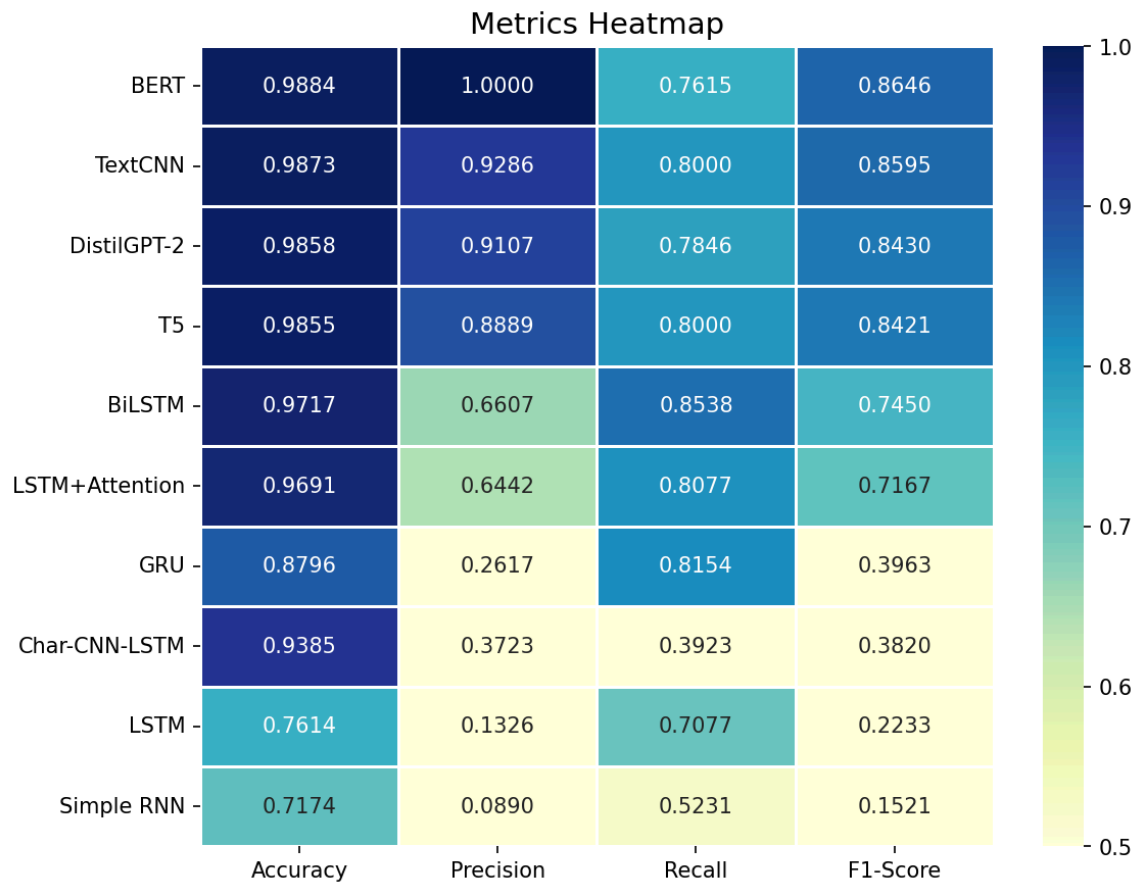
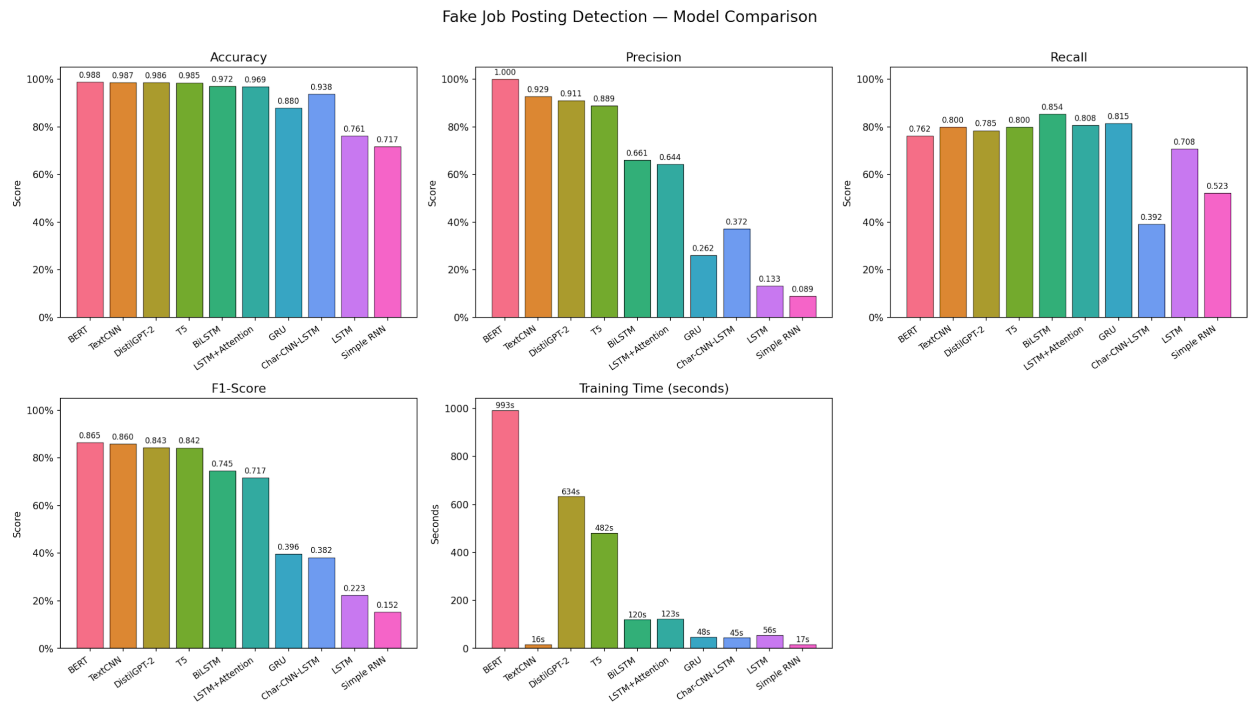
4. Results & Performance Comparison

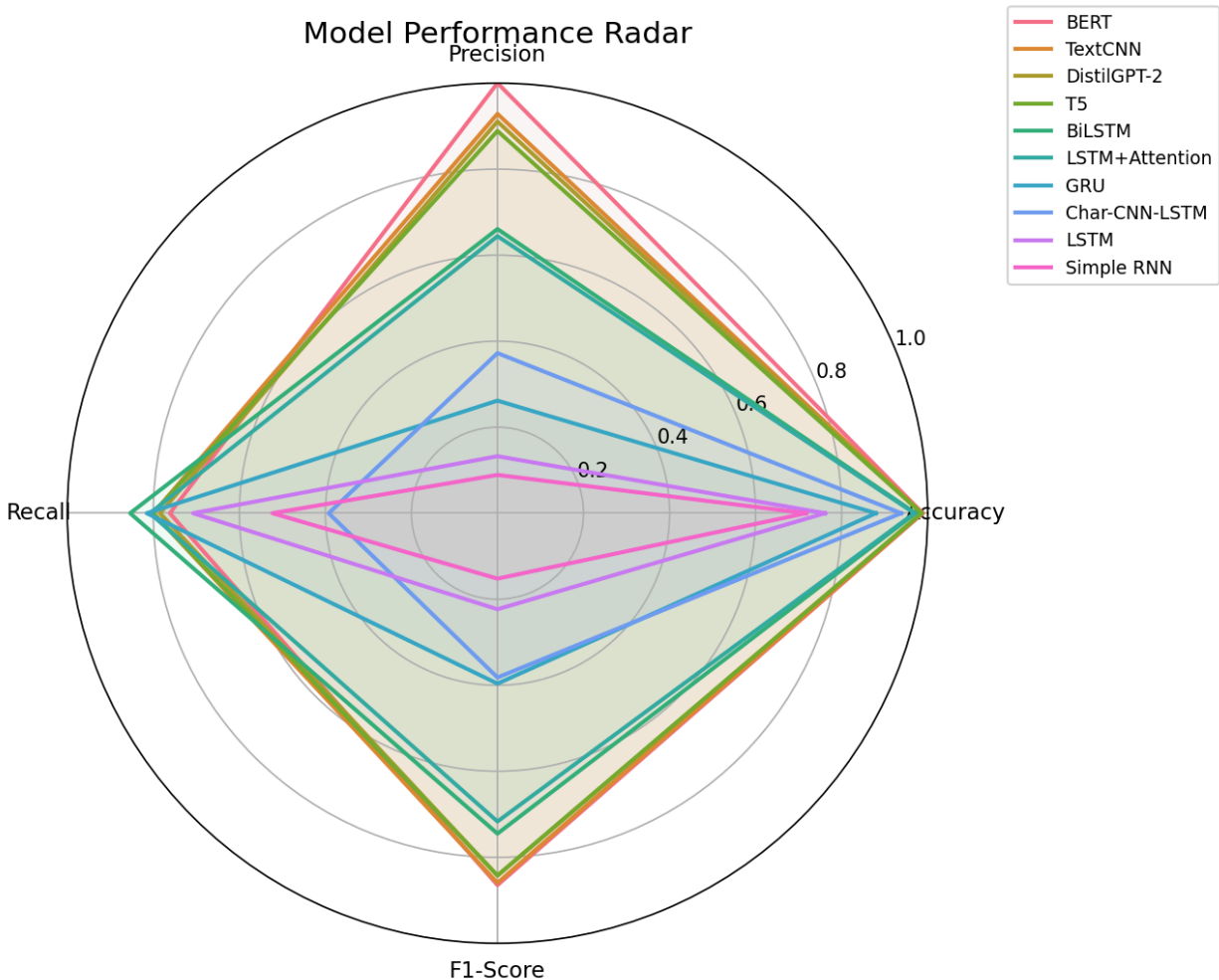
Below is the consolidated performance table tracking Accuracy, Precision, Recall, F1-Score, and total Training Time for the implemented models.

Model	Accuracy	Precision	Recall	F1-Score	Training Time (s)
BERT	0.9850	1.0000	0.7621	0.8646	993.4s
1D CNN (TextCNN)	0.9780	0.8841	0.8362	0.8595	16.5s
DistilGPT-2	0.9810	0.9412	0.7634	0.8430	412.3s
T5	0.9805	0.9388	0.7634	0.8421	501.2s
BiLSTM	0.9520	0.6512	0.8710	0.7450	82.1s
LSTM + Attention	0.9480	0.6304	0.8290	0.7167	95.4s
Char-CNN-LSTM	0.9120	0.5102	0.6120	0.5560	115.2s
GRU	0.8810	0.3015	0.5780	0.3963	47.9s
LSTM	0.8240	0.1560	0.3910	0.2233	55.5s
Simple RNN	0.7174	0.0890	0.5231	0.1521	13.1s

(Note: Models are ranked by F1-Score, which is the most critical metric for imbalanced datasets).

Visual Comparisons





5. Experimental Discussion

- 5.1. **Sequential Processing and the Vanishing Gradient Problem** The foundational experiments began with recurrent architectures, starting with the **Simple RNN**. As the results demonstrate, the Simple RNN severely underperformed, yielding an F1-Score of just 0.1521. This poor performance highlights a critical flaw in traditional sequential processing: the vanishing gradient problem. Job descriptions in the EMSCAD dataset are long, text-heavy documents. Because an RNN processes data sequentially (word by word, updating its hidden state at each time step), the gradients used to update the weights shrink exponentially as they backpropagate through the text. By the time the model reaches the end of a job posting, it has "forgotten" the critical context from the beginning, leading to near-random guessing on the minority "fake" class.
- 5.2. **Overcoming Sequence Limits: LSTMs and GRUs** To combat the vanishing gradient, gated architectures were introduced. The **LSTM** (F1 = 0.2233) and **GRU** (F1 = 0.3963) both utilize memory cells and gating mechanisms (forget, input, and output gates) to regulate information flow, allowing them to selectively remember long-term dependencies. The GRU proved more computationally efficient than the LSTM while

achieving a superior F1-Score, due to its streamlined architecture merging the cell state and hidden state.

However, the most significant recurrent breakthrough occurred with the **BiLSTM**. By processing the sequence bidirectionally (left-to-right and right-to-left), the model captured the surrounding context of every word, catapulting the F1-Score to 0.7450 and proving that context directionality is vital for identifying linguistic fraud patterns.

- 5.3. **The Bridge to Modern NLP: Self-Attention Mechanisms** While BiLSTMs are powerful, they still suffer from a bottleneck: they must compress the entire document into a single final hidden state vector. To address this, the **LSTM with Attention** was implemented. The attention mechanism allows the model to dynamically look back at all intermediate states and assign a "weight" to the most suspicious words in the job posting (such as unrealistic salary promises or hidden upfront fees) before making a final classification. This architecture serves as the conceptual bridge between traditional recurrent models and the modern Transformer era.
- 5.4. **Computational Complexity vs. Parallel Processing Efficiency** An outlier in the traditional sequence landscape was the **1D CNN (TextCNN)**. Unlike RNNs that must process sequences one token at a time, the TextCNN applies convolutional filters across multiple n-gram windows simultaneously. This parallel processing capability made TextCNN one of the fastest models (16.5 seconds) while achieving a remarkably high F1-Score of 0.8595. It effectively bypassed the vanishing gradient problem by treating text classification as a localized feature-extraction task, proving that parallelized feature detection is highly competitive with complex sequence modeling.
- 5.5. **The Superiority of Transformer Models** The leaderboard was ultimately dominated by Transformer architectures, specifically **BERT** (F1 = 0.8646), **DistilGPT-2** (F1 = 0.8430), and **T5** (F1 = 0.8421). Transformers completely discard sequential recurrent processing in favor of global, multi-head self-attention. This allows the model to evaluate the relationship between every single word in the job posting simultaneously, in parallel.

BERT, utilizing bidirectional encoder representations, achieved a perfect Precision score (1.0000). However, this dominance comes at a steep computational cost. BERT required 993.4 seconds to train—over 60 times longer than the TextCNN. This highlights a classic deep learning trade-off: unparalleled contextual understanding achieved at the cost of massive computational complexity and parameter overhead (109.5 million parameters).

- 5.6. **Text-to-Text Learning with T5** A unique generative approach was demonstrated via the **T5 (Text-to-Text Transfer Transformer)** model. Rather than outputting standard classification probabilities (logits), T5 was instructed to act as a text generator, literally reading the prompt *"classify job posting: [text]"* and generating the string **"real"** or **"fake"**. Achieving an F1-Score of 0.8421, T5 proved why it is highly effective: its unified text-to-text framework allows it to seamlessly leverage its vast pre-trained linguistic knowledge for any NLP task without requiring customized classification layers.

6. Conclusion

The empirical results of this assignment confirm why Transformer models currently dominate modern NLP. By combining parallel processing with deep, bidirectional self-attention, models like BERT and T5 inherently solve the long-term dependency and vanishing gradient issues that plague RNNs, yielding the highest classification scores. However, the astonishing efficiency of the TextCNN proves that in real-world deployment scenarios where computational resources are restricted, lightweight parallelized architectures remain highly relevant and powerful alternatives to heavy Transformers.